

Ripasso if, if else e Loops

Nei diagrammi a blocchi la condizione veniva scritta con un rombo dal quale partivano due rami:

1. Un ramo true
2. Un ramo false

Da d'ora in poi vengono definite nelle istruzioni if, if else, else



Python

```
1  n:int= int(input("Digitare un numero:")) #La funzione int ti
   converge tutto quello che si trova nell'argomento come un numero
   integer
2  if n ==5:
3      print(f"{n} è uguale a 5") #il print è stata indentata perché
   fa parte del blocco if e quindi viene eseguita se il blocco if
   risulta vero
4  else:
5      if n%2==0:
6          print(f"{n} non è uguale a 5 ma è pari")
7      '''
8  questo blocco di codice posso anche scriverlo così:
9      '''
10 if n == 5:
11     print(f"{n} è uguale a 5")
12 elif n%2==0:
13     print(f"{n} non è uguale a 5 ma è pari")
14 else:
15     print(f"{n} non è uguale a 5 ma è dispari")
16 #Quindi prima vado a valutare la mia condizione if se è vera mi
   printa il primo print, se è falsa vado a valutare la condizione
   elif; se questa condizione è vera mi esegue le istruzioni
   previste per questa condizione.
17 #Se questa condizione è falsa valuto l'else che va a chiudere lo
   statement condizionale.
18 #Al posto dell'elif posso annidare anche diversi if, è la stessa
   cosa.
19
20
21
```

I cicli ci permettono di ripetere in maniera automatica delle istruzioni che si ripetono, in python abbiamo 2 tipologie:

1. While

C'è una condizione che determina la ripetizione del ciclo fin tanto che la condizione sia vera, ma posso anche non entrare nel ciclo se la condizione non è stata verificata



Python

```
1  print("While")
2  #Per lavorare con un while ho bisogno di definire un contatore
3  i:int=1
4  while i<=5:
5      print("Hello World")
6      i=i+1
7
8  print("FOR")
9  for i in range(5):
10     print("Hello World")
```

Da notare come ho utilizzato lo stesso contatore: tuttavia nel while il contatore nasce e cresce nel ciclo mentre con il for invece

IL while è utile quando non so quante iterazioni devo fare, mentre se so quante iterazioni devo fare utilizzo il ciclo for:

questo ciclo vuole 3 regole

1. un valore di inizio
2. un valore finale
3. un'intervallo di valori(range) per incrementare il contatore

Di fatti la funzione in range mi restituisce in output il numero delle iterazioni che ho messo in argomento



Python

```

1  for i in range (1,11): #mi stampa in output tutti i valori da 1 a
    10 perche mi esclude l'ultimi numero
2      print(i)
3
4  for i in range(1,11,2): #Quest'ultimo valore mi dice quante unita
    devo implementare il contatore
5      print(i)

```

Il ciclo for viene usato per lavorare con le liste:

mettiamo ho una lista vuota e posso riempirla con un ciclo, o posso aumentare gli elementi della lista o modificarli:



Python

```

1  lista:list = [1,2,3,4,5]
2
3  #1 modo
4  print("modo 1")
5  for i in range(len(lista)): #la funzione len mi restituisce la
    lunghezza della lista
6      print(lista[i]) #èrendo il valore che assume i e lo uso per
    accedere alle posizioni degli elementi della lista
7
8  #2 modo
9  for item in lista:
10     print(item)
11  #In questo caso item diventa un elemento della lista, perche in
    questo caso vado a chiedere il valore stesso e quindi per ogni
    elemento della mia lista lo stampo in output
12

```

Le istruzioni break e continue:



Python

```

1  for i in range(1,11):
2      if i%2==0:
3          continue #mi interrompe l'iterazione corrente e mi skippa
    all'iterazione dopo
4      print(i)
5      #in questo caso mi stampo solo i numeri dispari perché mi
    skippa l'if
6  for i in range(1,11):

```

```
7     x:int=int(input("digita un numero"))
8     if x%2==0
9         break #Interrompe il ciclo e ti fa finire il ciclo quindi in
           questo caso per ogni numero dispari in questo caso il ciclo si
           interromperà
10
11     print("DAJE")
```